คู่มือสร้างและติดตั้งระบบทีละขั้น

PKRU Electronic Certificate

ระบบออกและตรวจสอบเกียรติบัตรอิเล็กทรอนิกส์

คู่มือสำหรับนำโค้ดในโปรเจกต์มาติดตั้งบนเครื่องใหม่ เริ่มใช้งานกับหน่วยงาน และเตรียมย้ายขึ้นเซิร์ฟเวอร์ โดยใช้ Docker เป็นวิธีหลัก

ฉบับวันที่ 12 กันยายน 2569 (2026-09-12)
อ้างอิงโค้ดในโปรเจกต์ Electronic Certificate-demosd ณ วันที่จัดทำ

> เป้าหมาย: เปิดเว็บได้ สร้างผู้ดูแลได้ ออกเกียรติบัตรได้ ตรวจ QR ได้ และสำรองข้อมูลได้ โดยใช้ PostgreSQL 18 ฐานข้อมูล pkru_certificate และ schema public ชุดเดียว

เริ่มอ่านตามลำดับ

ขั้น	เนื้อหา	หน้า
1-3	เตรียมเครื่อง ตั้งค่า และเปิด Docker	2-4
4-6	สร้างแอดมิน แม่แบบ และออกเกียรติบัตร	5-7
7	เข้าใจฐานข้อมูลและตำแหน่งรูป	8
8-9	เปิดลิงก์สาธารณะและจัดการ QR	9-10
10-11	สำรอง อัปเดต และย้ายเซิร์ฟเวอร์	11-12
12	แก้ปัญหาและตรวจรับระบบ	13
ภาคผนวก	โครงสร้างโค้ดและเอกสารอ้างอิง	14

ขอบเขตของคู่มือ

คำสั่งหลักใช้ PowerShell บน Windows และรันจากโฟลเดอร์รากของโปรเจกต์ ให้คัดลอกคำสั่งทั้งบล็อก เครื่องหมาย ` ท้ายบรรทัดคือตัวต่อคำสั่งของ PowerShell อย่าเติมช่องว่างหลังเครื่องหมายนี้ บน Linux ให้รวมคำสั่ง Docker เป็นบรรทัดเดียวโดยตัดตัวต่อบรรทัดออก ส่วนสคริปต์ .ps1 ต้องใช้ PowerShell

เอกสารนี้เป็นคู่มือติดตั้งจากโค้ดที่มีอยู่ ไม่ใช่ผลรับรองความปลอดภัยหรือรายงานทดสอบบนเครื่องใหม่ รหัสผ่านและข้อมูลส่วนตัวจริงไม่ถูกใส่ในเอกสารนี้

1. เตรียมเครื่องและเข้าใจสิ่งที่จะติดตั้ง

1.1 เตรียมเครื่อง

1. ติดตั้ง Docker Desktop บน Windows ซึ่งรวม Docker Compose ไว้แล้ว และเปิดใช้งาน Linux containers
2. เปิด Docker Desktop จน Docker Engine พร้อมทำงาน
3. เตรียมโฟลเดอร์โปรเจกต์ให้ครบ รวมไฟล์ที่ขึ้นต้นด้วยจุด เช่น .env.docker.example และ .dockerignore
4. ใช้อินเทอร์เน็ตระหว่าง build ครั้งแรก เพื่อดาวน์โหลด image และไลบรารี
5. เปิด PowerShell และตรวจคำสั่งต่อไปนี้

```
docker version
docker compose version
```

ผลที่ควรได้: แสดงข้อมูล Docker Client/Server และรุ่น Docker Compose หากมีเฉพาะ Client หรือแจ้งติดต่อ daemon ไม่ได้ ให้เปิด Docker Desktop ให้พร้อมก่อน

1.2 เทคโนโลยีที่โค้ดกำหนด

ส่วน	เทคโนโลยีและหน้าที่
เว็บและ API	Python 3.13, FastAPI, Uvicorn
หน้าจอ	HTML, CSS, JavaScript
ฐานข้อมูล	PostgreSQL 18, SQLAlchemy, Alembic
ภาพเกียรติบัตร	Pillow, ฟอนต์ไทย, RAQM/HarfBuzz, PyThaiNLP
QR	qrcode สร้าง QR; zxing-cpp อ่านและตรวจ QR
Excel	openpyxl อ่านรายชื่อจาก .xlsx
เข้าสู่ระบบ	JWT และ Argon2 สำหรับแฮชรหัสผ่าน

รุ่นของ Python และ PostgreSQL กำหนดใน Dockerfile/compose.yaml ส่วนรุ่นไลบรารีกำหนดใน requirements.txt

1.3 Docker ช่วยอย่างไร

Docker image รวมโปรแกรมและไลบรารีที่จำเป็น ส่วน container คือโปรแกรมที่กำลังทำงาน ข้อมูลถาวรอยู่ใน volumes และโฟลเดอร์สำรอง จึงสามารถสร้าง web container ใหม่เพื่ออัปเดตโค้ดโดยใช้ข้อมูลเดิมได้

เมื่อติดตั้งด้วย Docker ไม่ต้องติดตั้ง Python หรือ PostgreSQL บน Windows เพิ่มเพื่อให้เว็บทำงาน pgAdmin เป็นเพียงเครื่องมือเสริมสำหรับดูฐานข้อมูล

2. เปิดโฟลเดอร์และตั้งค่าครั้งแรก

2.1 อยู่ในโฟลเดอร์ที่ถูกต้อง

ตัวอย่างพาธบนเครื่องเจ้าของโปรเจกต์ ให้เปลี่ยนตามที่เก็บโค้ดบนเครื่องของคุณ

```
Set-Location 'D:\ENDPRO\Electronic Certificate-demosd'
Get-Item compose.yaml, Dockerfile, requirements.txt, sql/schema.sql
```

ไฟล์ทั้งสี่ต้องมีอยู่ หากไม่พบ ให้ตรวจว่าไม่ได้เปิดอยู่ในโฟลเดอร์ docs หรือ static

2.2 สร้างไฟล์ตั้งค่าสำหรับ Docker

```
if (-not (Test-Path -LiteralPath '.env.docker')) {
    Copy-Item .env.docker.example .env.docker
}
notepad .env.docker
```

ถ้ามีไฟล์อยู่แล้ว ให้แก้ไฟล์เดิม คำสั่งตัวอย่างนี้จะไม่คัดลอกทับไฟล์ที่มีอยู่

2.3 สร้างรหัสลับสองชุด

รันคำสั่งนี้สองครั้ง แล้วนำผลคนละชุดใส่ POSTGRES_PASSWORD และ SECRET_KEY ห้ามใช้ค่าตัวอย่างสั้น ๆ แทนค่าสุ่ม

```
docker run --rm python:3.13-slim-bookworm `
  python -c "import secrets; print(secrets.token_hex(32))"
```

ตัวแปรใน .env.docker	ค่าที่ต้องกำหนด
POSTGRES_PASSWORD	ค่าสุ่มชุดแรก ใช้กับฐานข้อมูล
SECRET_KEY	ค่าสุ่มชุดที่สอง ใช้ลงลายเซ็น JWT
POSTGRES_USER	certificate_app
POSTGRES_DB	pkru_certificate
PUBLIC_BASE_URL	http://localhost:8000 สำหรับเริ่มในเครื่อง
APP_BIND_HOST / APP_PORT	127.0.0.1 / 8000
QR_UPDATE_ON_STARTUP	ค่าเริ่มต้นใน Compose คือ true; เพิ่มบรรทัดนี้ได้หากต้องเปลี่ยน

รหัสผ่านฐานข้อมูลไม่ใช่รหัสผ่านแอดมินเว็บ การสร้างบัญชีแอดมินอยู่ในขั้นที่ 4 เก็บ .env.docker ไว้ส่วนตัวและไม่ส่งเข้า Git หากใช้ฐานข้อมูลเดิมอยู่ การเปลี่ยน POSTGRES_PASSWORD ในไฟล์อย่างเดียวจะไม่เปลี่ยนรหัสผ่านภายใน PostgreSQL ให้ตาม

3. สร้างและเปิดระบบด้วย Docker

3.1 Build และเปิดบริการ

หลังบันทึก .env.docker แล้ว ให้รัน:

```
docker compose --env-file .env.docker up -d --build
docker compose --env-file .env.docker ps -a
```

ครั้งแรกอาจใช้เวลาสำหรับดาวน์โหลด image และติดตั้งไลบรารี คำสั่งเดียวนี้สร้างบริการตาม compose.yaml

3.2 อ่านสถานะบริการ

บริการ	หน้าที่	สถานะที่ควรพบ
db	PostgreSQL 18	running / healthy
migrate	เตรียมและอัปเกรดตาราง	Exited (0) หลังทำงานสำเร็จ
web	เว็บ FastAPI	running / healthy
backup	สำรองฐานข้อมูลและไฟล์	healthy หลังสำรองครั้งแรกสำเร็จ
scheduler	งานบำรุงรักษาตามเวลา	เป็น profile maintenance ไม่เปิดตามค่าเริ่มต้น

migrate ที่ Exited (0) เป็นผลปกติ เพราะเป็นงานครั้งเดียว ไม่ใช่บริการที่ต้องทำงานค้างตลอด

3.3 ตรวจหน้าเว็บ

เปิด http://localhost:8000 ในเบราว์เซอร์ แล้วตรวจสถานะ:

```
Invoke-RestMethod http://localhost:8000/healthz
```

ผลที่ควรได้: status เป็น ok ซึ่งหมายถึง endpoint ตรวจสุขภาพติดต่อฐานข้อมูลได้

หากเว็บยังไม่ขึ้น ให้ดูเหตุผลจาก log:

```
docker compose --env-file .env.docker logs --tail 100 web migrate
```

ฐานข้อมูลถูกสร้างเมื่อไร

scripts/initialize_database.py ตรวจ schema public หากยังไม่มีตารางจะอ่าน sql/schema.sql จากนั้นเรียก Alembic อัปเกรดถึง head หากมีฐานข้อมูลเดิมที่มีประวัติ Alembic จะใช้ migration แทนการสร้างตารางซ้ำ

ไม่ต้องสร้าง pkru_certificate_test หรือ schema สำหรับทดลองเพื่อใช้งานระบบนี้

4. สร้างแอดมินและหน่วยงานแรก

4.1 สร้างผู้ดูแลระบบครั้งแรก

```
docker compose --env-file .env.docker run --rm --no-deps web `
  python scripts/create_admin.py --username admin
```

คำสั่งจะถามรหัสผ่านโดยซ่อนสิ่งที่พิมพ์ ให้กำหนดอย่างน้อย 12 ตัวอักษร มีตัวพิมพ์ใหญ่ ตัวพิมพ์เล็ก ตัวเลข และสัญลักษณ์

ผลที่ควรได้: สร้างบัญชีผู้ดูแลสำเร็จ หากระบบแจ้งว่ามีผู้ดูแลแล้ว ให้ใช้บัญชีเดิม สคริปต์นี้ไม่ได้มีไว้สร้างแอดมินซ้ำหรือรีเซ็ตรหัสผ่านเดิม

4.2 เข้าสู่ระบบ

1. พิมพ์ http://localhost:8000/login ในช่องที่อยู่ของเบราว์เซอร์
2. กรอกชื่อผู้ใช้และรหัสผ่านที่เพิ่งสร้าง
3. ตรวจว่าพบเมนูของผู้ดูแล เช่น ภาพรวม หน่วยงาน และบันทึกการใช้งาน

หน้าตรวจสอบสาธารณะไม่มีปุ่มเข้าสู่ระบบตามการออกแบบ ให้เปิด /login โดยตรง

4.3 สร้างหน่วยงาน

1. เปิดเมนู **หน่วยงาน**
2. เพิ่มหน่วยงาน กรอกชื่อและข้อมูลที่แบบฟอร์มกำหนด พร้อมบัญชีแรกของหน่วยงาน
3. บันทึกแล้วตรวจว่าหน่วยงานอยู่ในสถานะใช้งาน
4. ออกจากบัญชีแอดมิน แล้วเข้าสู่ระบบด้วยบัญชีหน่วยงาน
5. เปิดเมนู **แม่แบบ** และตรวจว่ามีแม่แบบ **ต้นฉบับ** พร้อมใช้

บัญชีแอดมินใช้ดูแลหน่วยงานและบัญชี ส่วนบัญชีหน่วยงานใช้จัดแม่แบบและออกเกียรติบัตร ข้อมูลเชื่อมกันด้วย organization_id

4.4 เพิ่มผู้ใช้งานในหน่วยงาน

กลับเข้าบัญชีแอดมิน แล้วใช้ส่วนจัดการบุคลากรของหน่วยงาน เพื่อเพิ่มบัญชี เปิด/ปิดบัญชี หรือตั้งรหัสผ่านใหม่ ไม่ต้องสร้างฐานข้อมูลแยกให้ผู้ใช้แต่ละคน

> หากลองล็อกอินผิดหลายครั้ง ระบบอาจให้รอ 15 นาทีตามค่าที่ตั้งไว้ การปิดเปิดเว็บไม่ล้างตัวนับ เพราะเก็บไว้ใน PostgreSQL

5. สร้างและปรับแม่แบบเกียรติบัตร

5.1 เริ่มจากต้นฉบับ

แม่แบบต้นฉบับเก็บใน assets/templates/original/ และรวมอยู่ใน Docker image หน่วยงานใหม่ได้รับแม่แบบนี้เสมอ หากต้องแก้ไขให้เลือก **ทำสำเนาเพื่อแก้ไข**

ต้นฉบับของระบบลบ ปิดใช้ หรือแก้ทับไม่ได้ ส่วนแม่แบบที่สร้างเองและสำเนาสามารถลบได้ โดยไม่ทำให้เกียรติบัตรที่ออกไปแล้วถูกลบตาม

5.2 เพิ่มพื้นหลังของหน่วยงาน

1. เปิดเมนู **แม่แบบ** แล้วเลือกเพิ่มแม่แบบ
2. อัปโหลดพื้นหลัง PNG หรือ JPG/JPEG ขนาดไม่เกิน 10 MB
3. ตั้งชื่อแม่แบบให้อธิบายงาน เช่น ชื่อกิจกรรมหรือปีที่ใช้งาน
4. เปิดแก้ไขตำแหน่ง เพื่อจัดบล็อกข้อมูลบนพื้นหลัง

5.3 จัดบล็อกข้อความ ลายเซ็น และ QR

- เพิ่มบล็อกชื่อผู้รับ หัวข้อ วันที่ หรือข้อมูลเพิ่มเติมตามแบบ
- ลากบล็อกเพื่อย้ายตำแหน่ง และปรับขนาดตามพื้นที่
- ใช้คำสั่งจัดกึ่งกลางและเส้นช่วยจัดแนวในตัวแก้ไข
- เพิ่มบล็อกลายเซ็นได้หลายบล็อก แต่ละบล็อกมีชื่อและตำแหน่งผู้ลงนามของตนเอง
- กำหนดฟอนต์ ขนาดตัวอักษร สี และการจัดแนวตามเครื่องมือที่มี
- วาง QR และข้อความรหัสตรวจสอบให้อ่านได้ โดยรหัสสามารถเป็นบล็อกข้อความใต้ QR และปรับสีได้

ตำแหน่งบล็อกเก็บเป็นสัดส่วนของหน้า จึงย้ายแม่แบบไปใช้กับขนาดภาพต่างกันได้ แต่ต้องตรวจความอ่านง่ายและขนาด QR เมื่อพิมพ์จริง

5.4 ตรวจตัวอย่างก่อนบันทึก

1. ตรวจชื่อที่ยาว หัวข้อหลายบรรทัด และตำแหน่งลายเซ็น
2. ตรวจว่า QR ไม่ทับข้อความหรือขอบลวดลาย
3. ตรวจข้อความรหัสใต้ QR ว่าตัดกับสีพื้นหลังชัดเจน
4. บันทึกแม่แบบ แล้วเลือกแม่แบบนี้ในหน้ารายบุคคล

ผลที่ควรได้: พรีวิวแสดงตำแหน่งและภาษาไทยถูกต้อง ไม่ตัดข้อมูลสำคัญ และมีพื้นที่สำหรับผู้ลงนามครบ

6. ทดลองใช้งานกับเกียรติบัตรหนึ่งใบ

6.1 ออกเกียรติบัตรรายบุคคล

1. เข้าสู่ระบบด้วยบัญชีหน่วยงาน แล้วเปิด **ออกเกียรติบัตรรายบุคคล**
2. เลือกแม่แบบ กรอกชื่อผู้รับ หัวข้อ/กิจกรรม และวันที่ออก
3. เลือกประเภทผู้รับ หากเป็นบุคคลภายนอก ให้กรอกหน่วยงานภายนอก
4. กรอกช่องเพิ่มเติมที่แม่แบบกำหนด หากมีช่วงวันต้องกรอกทั้งวันเริ่มและวันสิ้นสุด หรือเว้นว่างทั้งคู่เมื่อไม่บังคับ
5. ใส่ลายเซ็น ชื่อ และตำแหน่งให้ครบทุกบล็อกที่แม่แบบต้องใช้
6. ตรวจข้อมูล ทำเครื่องหมายยืนยัน แล้วกดยืนยันการออกเกียรติบัตร
7. ดาวน์โหลด PNG และเปิดหน้าตรวจสอบด้วยรหัสที่ระบบสร้าง

วันเริ่มและวันสิ้นสุดเป็นวันเดียวกันได้ แต่วันสิ้นสุดต้องไม่ก่อนวันเริ่ม หากข้อมูลไม่ผ่าน ระบบจะแจ้งชื่อช่องและเหตุผลที่แก้ไขได้

> การกดยืนยันสร้างข้อมูลเกียรติบัตรจริงในฐานข้อมูล ใช้ข้อมูลที่ได้รับอนุญาตและตรวจทานก่อนยืนยัน ใบที่ออกแล้วใช้การเพิกถอนเมื่อจำเป็น

6.2 ตรวจสอบจากหน้าสาธารณะ

เปิดหน้าหลัก กรอกรหัสตรวจสอบ แล้วดูชื่อผู้รับ หน่วยงาน สถานะ และภาพ ดาวน์โหลด PNG/JPG/PDF ได้ตามฟังก์ชันของหน้าเว็บ

ถ้า QR ยังใช้ localhost การสแกนจากโทรศัพท์ภายนอกเครื่องจะยังใช้ไม่ได้ ให้ทำขั้นที่ 8-9 ก่อนทดสอบจากเครือข่ายอื่น

6.3 ออกแบบกลุ่มจาก Excel

1. เตรียมไฟล์ .xlsx แถวแรกเป็นชื่อคอลัมน์ และหนึ่งแถวต่อผู้รับ
2. เปิด **ออกเกียรติบัตรแบบกลุ่ม** เลือกแม่แบบและวันที่
3. อัปโหลดไฟล์และจับคู่คอลัมน์กับช่องของแม่แบบ
4. ตรวจรายการที่ผิด แก้ข้อมูล และใส่ลายเซ็นตามที่ระบบร้องขอ
5. ยืนยันประมวลผล แล้วติดตามใน **รายการประมวลผล**
6. ตรวจจำนวนสำเร็จ/ล้มเหลว และดาวน์โหลด ZIP เมื่อพร้อม

ระบบรองรับไฟล์ Excel สูงสุด 10 MB, 5,000 แถวข้อมูล และ 30 คอลัมน์ ไม่ใช้เมนู CSV หรือการวางข้อมูลจาก Excel แบบเดิม

7. ฐานข้อมูลใช้ไฟล์ไหน และรูปอยู่ที่ใด

7.1 ใช้ฐานข้อมูลหลักชุดเดียว

รายการ	ค่าที่ใช้
ระบบฐานข้อมูล	PostgreSQL 18
ชื่อฐานข้อมูล	pkru_certificate
Schema ของแอป	public
สร้างตารางใหม่	sql/schema.sql
โครงสร้างในโค้ด	models.py
อัปเกรดฐานข้อมูลเดิม	alembic/ ผ่านบริการ migrate

มีตารางของระบบ 12 ตาราง เช่น organizations, users, templates, certificates, batches และ audit_logs พร้อมตาราง alembic_version สำหรับบันทึกเวอร์ชันโครงสร้าง

คำว่า public ใน PostgreSQL คือชื่อ schema ไม่ได้หมายความว่าฐานข้อมูลเปิดให้ทุกคนอ่านผ่านอินเทอร์เน็ต

7.2 แยกข้อมูลตารางกับไฟล์ภาพ

ข้อมูล	ที่เก็บขณะใช้ Docker
บัญชี แฮชรหัสผ่าน หน่วยงาน รายการเกียรติบัตร	volume pkru-certificate_postgres_data
ภาพเกียรติบัตร PNG	/app/storage/generated/certificates
พื้นหลังแม่แบบและฟอนต์อัปโหลด	/app/storage/uploads/templates และ fonts
ไฟล์ข้างต้นใน storage	volume pkru-certificate_certificate_storage
ชุดสำรองข้อมูล	โฟลเดอร์ backups/ ในโปรเจกต์ตามค่าตั้ง
แม่แบบต้นฉบับประจำระบบ	assets/templates/original/ ใน image

ฐานข้อมูลเก็บรายละเอียดและพาธภาพ ส่วนภาพจริงอยู่ใน volume ของ storage ไฟล์ JPG/PDF สร้างจาก PNG เมื่อร้องขอดาวน์โหลด จึงต้องสำรองทั้งฐานข้อมูลและ storage ให้สอดคล้องกัน

7.3 ข้อควรจำเมื่อติดตั้งฐานข้อมูล

เมื่อติดตั้งตามคู่มือนี้ ให้บริการ migrate เตรียมตาราง ไม่ต้องเปิด pgAdmin เพื่อรัน SQL ซ้ำ และไม่ต้องสร้างฐานข้อมูลเพิ่มเติม

sql/schema.sql เป็นโครงสร้างเริ่มต้น ไม่ใช่ข้อมูลสำรอง ฐานข้อมูลที่ใช้งานแล้วให้อัปเกรดผ่าน Alembic ตามขั้นที่ 10 และใช้ชุดสำรองเมื่อต้องกู้ข้อมูล

8. เปิดลิงก์สาธารณะสำหรับทดลอง

8.1 เลือกวิธีให้ผู้รับเข้าถึง

- ใช้ในเครื่อง: http://localhost:8000
- ทดลองจากภายนอก: Cloudflare Quick Tunnel
- ใช้งานระยะยาว: โดเมนถาวรและ HTTPS บนเซิร์ฟเวอร์

localhost หมายถึงเครื่องของผู้เปิดลิงก์ ผู้รับจึงใช้ localhost ของผู้พัฒนาโดยตรงไม่ได้

8.2 ตรวจว่ามี Tunnel เดิมหรือไม่

```
docker ps -a --filter name=pkru-certificate-tunnel
```

หากมี container เดิมที่หยุดอยู่ ให้เปิดด้วย:

```
docker start pkru-certificate-tunnel
```

หากยังไม่มี container ชื่อนี้ จึงใช้คำสั่งสร้างด้านล่าง คำสั่งเปิดให้คนที่มีลิงก์เข้าถึงเว็บจากภายนอกได้:

```
docker run -d --name pkru-certificate-tunnel --restart unless-stopped `
  --network pkru-certificate_default cloudflare/cloudflared:latest `
  tunnel --no-autoupdate --protocol http2 --url http://web:8000
```

ชื่อ network อ้างอิง project name pkru-certificate ใน compose.yaml หากเปลี่ยนชื่อ project ต้องตรวจชื่อ network ให้ตรงก่อน

8.3 อ่าน URL ที่ได้รับ

```
docker logs --tail 60 pkru-certificate-tunnel
```

มองหา URL ที่ลงท้ายด้วย trycloudflare.com แล้วเปิดทดสอบในเบราว์เซอร์ คำสั่ง logs ใช้หลังเริ่ม Tunnel หรือเมื่อต้องการดู URL/ปัญหาการเชื่อมต่อ ไม่ต้องรันทุกครั้งที่ออกเกียรติบัตร

Quick Tunnel สร้าง subdomain แบบสุ่มและมีไว้สำหรับทดลองพัฒนา จึงควรตรวจ URL จาก log เมื่อเริ่มใหม่ ไม่ใช้เป็นโดเมนถาวรของระบบจริง ดู คู่มือ Quick Tunnels ของ Cloudflare

ผลที่ควรได้: เปิดหน้าตรวจสอบผ่าน URL สาธารณะได้ จากนั้นนำ URL นี้ไปตั้งในขั้นที่ 9 เครื่องต้นทางและ Docker ต้องทำงานระหว่างให้บริการผ่าน Tunnel

9. ตั้ง URL ให้ QR ตรงกับเซิร์ฟเวอร์

9.1 เปลี่ยน URL ในไฟล์ตั้งค่า

เปิด .env.docker แล้วตั้ง PUBLIC_BASE_URL เป็น URL สาธารณะจริง ตัวอย่างรูปแบบ:

```
PUBLIC_BASE_URL=https://certificate.example.org
```

certificate.example.org เป็นชื่อแสดงรูปแบบเท่านั้น ต้องแทนด้วยโดเมนหรือลิงก์ Tunnel ของคุณ ไม่ต้องใส่ /?code= ต่อท้าย และไม่ใช้ URL ที่มีชื่อผู้ใช้ รหัสผ่าน query หรือ fragment

9.2 ตรวจล่วงหน้าก่อนแก้ภาพเดิม

หลังบันทึกไฟล์ตั้งค่า สามารถตรวจโดยไม่เขียนภาพได้:

```
docker compose --env-file .env.docker run --rm --no-deps web `
  python -m scripts.update_certificate_qr --dry-run
```

หากมี failed ให้ดูหมายเลขรายการและเหตุผล ภาพที่ตรวจไม่ผ่านจะไม่ถูกแทนที่อย่างเดา ๆ

9.3 ให้ web รับค่าใหม่

```
docker compose --env-file .env.docker up -d --no-deps `
  --force-recreate web
```

การ restart อย่างเดียวไม่รับค่า environment ที่เปลี่ยนใน .env.docker การสร้าง container ใหม่จะใช้ค่าใหม่โดยยังอ้างอิง volumes เดิม

เมื่อ QR_UPDATE_ON_STARTUP เป็น true ระบบเริ่มงานเบื้องหลังเพื่อตรวจและอัปเดต QR ใน PNG ที่เก็บไว้ หากปิดตัวเลือกนี้ สามารถสั่งเองได้:

```
docker compose --env-file .env.docker run --rm --no-deps web `
  python -m scripts.update_certificate_qr --apply
```

9.4 ตรวจผล

เปิดใบเก่าและใบใหม่จากระบบ แล้วสแกน QR ตรวจว่าเปิดโดเมนปัจจุบัน พร้อมรหัสเดิมถูกต้อง ระบบอัปเดตเฉพาะกรอบ QR และตรวจว่าพิกเซลนอกกรอบไม่เปลี่ยน

ไฟล์ที่ดาวน์โหลดหรือพิมพ์แจกไปแล้วไม่เปลี่ยนตามเซิร์ฟเวอร์ ต้องดาวน์โหลดใหม่ หรือคงโดเมนเดิมและตั้งการส่งต่อที่เหมาะสม การใช้โดเมนถาวรลดปัญหานี้

10. เปิด-ปิด สำรอง และอัปเดตระบบ

10.1 คำสั่งใช้งานประจำบน Windows

```
.\scripts\docker.ps1 status
.\scripts\docker.ps1 start
.\scripts\docker.ps1 stop
.\scripts\docker.ps1 logs
```

เลือกใช้ทีละคำสั่งตามงาน stop หยุดบริการโดยเก็บข้อมูล ส่วน logs ติดตาม log ของ web กด Ctrl+C เพื่อออก สคริปต์ restart เริ่มเฉพาะ web ใหม่ แต่ไม่รับ environment ที่แก้ไข

หากเครื่องไม่อนุญาตให้รัน .ps1 สามารถใช้ Docker Compose โดยตรง เช่น:

```
docker compose --env-file .env.docker ps -a
docker compose --env-file .env.docker stop
docker compose --env-file .env.docker start
```

10.2 สำรองข้อมูล

```
.\scripts\backup.ps1 status
.\scripts\backup.ps1 run
.\scripts\backup.ps1 verify
```

backup สำรองฐานข้อมูลและ storage อัตโนมัติ ค่าเริ่มต้นทุกวัน 03:00 น. เขตเวลา Asia/Bangkok และเก็บ 14 ชุด ตรวจสถานะจริงจากคำสั่ง status

verify ทดลองกู้คืนใน container PostgreSQL 18 ชั่วคราวที่แยกออก ไม่กู้ทับฐานข้อมูลใช้งานและไม่เพิ่มฐานข้อมูลถาวรให้โปรเจกต์ ควรมีสำเนาของชุดสำรองนอกเครื่องด้วย

10.3 อัปเดตโค้ดที่มีผลต่อเว็บและฐานข้อมูล

สำรองและตรวจชุดสำรองก่อน จากนั้นอัปเดตไฟล์โค้ดที่ได้รับตรวจสอบ แล้วรันตามลำดับ:

```
docker compose --env-file .env.docker stop web scheduler backup
docker compose --env-file .env.docker build
docker compose --env-file .env.docker run --rm --no-deps migrate
docker compose --env-file .env.docker up -d
```

หลังอัปเดตตรวจ /healthz, สถานะบริการ และการเปิดใบเดิม ถ้า migrate ล้มเหลวให้ตรวจ log และสาเหตุก่อน อย่าเปิดใช้งานต่อโดยข้ามขั้นตอนนั้น

ห้ามใช้ down -v หากต้องเก็บข้อมูลเดิม ตัวเลือกนี้ลบ volumes ซึ่งเก็บฐานข้อมูลและรูปของระบบ

11. เตรียมย้ายขึ้นเซิร์ฟเวอร์มหาวิทยาลัย

11.1 สิ่งที่ต้องตกลงกับผู้ดูแลเซิร์ฟเวอร์

1. เครื่องหรือ VM สำหรับ Docker และพื้นที่ถาวรของฐานข้อมูล รูป และสำรอง
2. โดเมนที่จะใช้ เช่น subdomain ของมหาวิทยาลัย และ HTTPS
3. ผู้ดูแล DNS, reverse proxy, firewall และสิทธิ์เข้าถึงเครื่อง
4. โฟลเดอร์สำรองนอกเครื่อง และวิธีกู้คืนเมื่อเกิดปัญหา
5. ช่วงเวลาหยุดระบบสำหรับย้ายข้อมูลและตรวจรับ

ใช้ PostgreSQL 18 ตามเวอร์ชันที่โค้ดและ image ของโปรเจกต์กำหนด

11.2 ลำดับการย้าย

1. สำรองฐานข้อมูลและ storage ของเครื่องเดิมให้เป็นชุดที่สอดคล้องกัน
2. นำโค้ดขึ้นเครื่องใหม่ พร้อมตั้ง .env.docker สำหรับเครื่องนั้น
3. เตรียม volumes/โฟลเดอร์ storage และกู้คืนทั้งฐานข้อมูลกับไฟล์ตามคู่มือ deployment.md
4. ตั้ง reverse proxy ให้โดเมน HTTPS ส่งมายัง web เช่น 127.0.0.1:8000 บน host
5. ตั้ง PUBLIC_BASE_URL เป็นโดเมนถาวร แล้วสร้าง web container ใหม่
6. ตรวจการอัปเดต QR ของภาพที่เก็บอยู่ และทดสอบเปิดจากเครือข่ายภายนอก
7. ตรวจชื่อผู้รับ หน่วยงาน จำนวนใบ และการดาวน์โหลดใบเก่าก่อนเปิดใช้งานจริง

หากกู้ข้อมูลเดิมแล้ว ให้ใช้บัญชีที่กู้มาด้วย ไม่ต้องสร้างแอดมินใหม่ และไม่รัน SQL สำหรับฐานข้อมูลใหม่ทับฐานข้อมูลที่กู้คืน

11.3 จุดที่ควรตรวจเพิ่มเติมก่อนเปิดวงกว้าง

- ระบบรีเซ็ตรหัสผ่านปัจจุบันยังไม่มีการยกเลิก JWT เดิมทันที
- Token อยู่ใน localStorage และยังควรเพิ่ม CSP/การป้องกันฝังหน้าเว็บ
- การรับ IP หลัง proxy ต้องกำหนด trusted proxy ให้ถูกต้อง มิฉะนั้นอาจใช้โควตาล็อกอินร่วมกัน
- ตรวจสิทธิ์ฐานข้อมูล การเข้ารหัสสำรอง HTTPS และการอัปเดตไลบรารีกับผู้ดูแลเครื่อง
- ผู้ที่มีรหัสตรวจสอบสามารถเข้าถึงข้อมูลและภาพตามหน้าสาธารณะได้ ควรใช้ข้อมูลที่เหมาะกับการเผยแพร่

รายการนี้เป็นประเด็นจากการตรวจโค้ด ไม่ได้ระบุว่าฟังก์ชันที่เสนอให้เพิ่มถูกพัฒนาแล้ว ภาพลายเซ็นในใบประกาศยังเป็นภาพ ไม่ใช่ลายเซ็นดิจิทัล PKI ใน PDF

12. แก้ปัญหาและตรวจรับการติดตั้ง

12.1 ปัญหาที่พบบ่อย

อาการ	วิธีตรวจและแก้
ติดต่อ Docker ไม่ได้	เปิด Docker Desktop และรอ Engine พร้อม ตรวจ docker version
localhost:8000 เปิดไม่ได้	ตรวจ compose ps -a และ log web/migrate; ตรวจพอร์ตชน
พอร์ต 8000 ถูกใช้	เปลี่ยน APP_PORT พร้อม PUBLIC_BASE_URL ให้ตรง แล้วสร้าง web ใหม่
รหัสฐานข้อมูลไม่ผ่าน	ตรวจว่าค่าตรงกับฐานข้อมูลเดิม การแก้ .env ไม่เปลี่ยนรหัสใน PostgreSQL
migrate ไม่มี alembic_version	หยุดตรวจโครงสร้างเดิมก่อนนำเข้า อย่าลบทิ้งหรือรัน SQL ทับ
มีแอดมินอยู่แล้ว	ใช้บัญชีเดิม สคริปต์สร้างแอดมินทำงานครั้งแรกเท่านั้น
ล็อกอินถูกจำกัด	รอตาม Retry-After; ตรวจว่าอยู่หลัง proxy เดียวกันหรือไม่
CERTIFICATE_FIELD_INVALID	อ่านชื่อช่องและสาเหตุ ตรวจช่วงวัน/ปีและข้อมูลเพิ่มเติม
QR เปิด URL เก่า	ตรวจ PUBLIC_BASE_URL, สร้าง web ใหม่ และดูผลอัปเดต QR
QR บนกระดาษยังเป็น URL เก่า	กระดาษไม่อัปเดต ต้องใช้โดเมนเดิมหรือออกไฟล์ให้ใหม่
backup ยังไม่ healthy	ตรวจ log backup และสิทธิ์เขียนโฟลเดอร์สำรอง
อักษรไทยแสดงผิด	ใช้ฟอนต์ที่มากับโปรเจกต์ ตรวจ RAQM และ build image ตาม Dockerfile

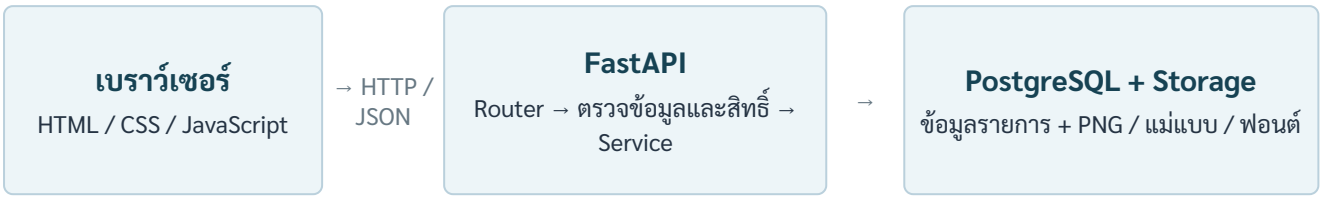
12.2 รายการตรวจรับ

- ☐ db และ web พร้อมใช้งาน; migrate จบด้วยรหัส 0
- ☐ เปิด /healthz แล้วได้ status: ok
- ☐ แอดมินเข้าสู่ระบบได้และสร้างหน่วยงานได้
- ☐ บัญชีหน่วยงานได้รับแม่แบบต้นฉบับ
- ☐ เพิ่มสำเนาแม่แบบและจัดลายเซ็นได้ครบ
- ☐ ออกเกียรติบัตรที่อนุมัติให้ใช้ตรวจรับได้ และดาวน์โหลดไฟล์ได้
- ☐ เปิดหน้าตรวจสอบด้วยรหัสแล้วข้อมูลตรงกับใบ
- ☐ สแกน QR ผ่านโดเมนสาธารณะจากเครื่องอื่นได้
- ☐ หยุดและเปิดบริการใหม่แล้วใบเดิมยังอยู่
- ☐ สำรองข้อมูลได้ และมีผลทดลองกู้คืนที่ตรวจสอบได้
- ☐ เก็บ .env.docker และชุดสำรองในพื้นที่จำกัดสิทธิ์

ช่องตรวจรับเว้นไว้สำหรับผู้ติดตั้งบันทึกผลจริง การจัดทำคู่มือนี้ไม่ได้รันขั้นตอนสร้างบัญชี เปลี่ยนข้อมูล หรืออัปเดตระบบแทนผู้ติดตั้ง

ภาคผนวก: โครงสร้างการสร้างระบบและไฟล์อ้างอิง

แผนภาพการทำงาน



หากต้องพัฒนาต่อ ควรเริ่มจากส่วนใด

ลำดับ	ไฟล์หรือโฟลเดอร์	หน้าที่
1	config.py, database.py	อ่านค่าตั้งและเชื่อม PostgreSQL
2	models.py, sql/schema.sql, alembic/	โมเดล ตาราง และประวัติอัปเกรด
3	schemas/, auth.py	ตรวจข้อมูล ยืนยันตัวตน และตรวจสิทธิ์
4	routers/	API สำหรับบัญชี แม่แบบ ออกใบ และตรวจสอบ
5	services/	ตรรกะสร้างภาพ ลายเซ็น นำเข้า และบำรุงรักษา QR
6	static/	หน้าจอและเครื่องมือแก้แม่แบบ
7	Dockerfile, compose.yaml, scripts/	Build ติดตั้ง แอดมิน สำรอง และดูแลระบบ
8	tests/	การตรวจพฤติกรรมของระบบสำหรับผู้พัฒนา

เอกสารและโค้ดที่ใช้ตรวจสอบคู่มือนี้

- README.md: วิธีติดตั้งหลัก SQL แม่แบบ และอัปเดต QR
- compose.yaml, Dockerfile, .env.docker.example: บริการ image และค่าตั้ง
- scripts/initialize_database.py, scripts/create_admin.py: เตรียมตารางและแอดมิน
- scripts/docker.ps1, scripts/backup.ps1: คำสั่งดูแลและตรวจสำรอง
- services/certificate_qr.py, services/qr_maintenance.py: การเปลี่ยน QR ใน PNG
- static/dashboard.html และ static/js/dashboard.js: เมนูและฟอร์มปัจจุบัน
- docs/deployment.md: คู่มือย้าย กู้คืน และนำขึ้นเซิร์ฟเวอร์
- docs/project-guide.md/.pdf: รายละเอียดระบบ ตาราง API และบัญชีไฟล์ฉบับก่อนหน้า; ใช้ README/โค้ดล่าสุดตรวจพฤติกรรม QR ที่เพิ่มภายหลัง

หลักสำคัญในการพัฒนาต่อ: ปรับโมเดลพร้อม migration เมื่อต้องเปลี่ยนฐานข้อมูล รักษาเงื่อนไขแยกหน่วยงาน ทดสอบพฤติกรรมที่เปลี่ยน สำรองข้อมูลก่อนอัปเกรด และอย่าเก็บข้อมูลสำคัญไว้เฉพาะในชั้นเขียนชั่วคราวของ container